

Transactions

COSC 304 – Introduction to Database Systems

Transaction Management Overview

The database system must ensure that the data is ***consistent***.

Two challenges in preserving consistency:

- 1) Must handle ***failures*** of various kinds (hardware, crashes).
- 2) Must support ***concurrent execution*** of multiple transactions and guarantee that concurrency does not cause inconsistency.

A ***transaction*** is an ***atomic*** program that executes on the database and preserves the consistency of the database.

- The input to a transaction is a consistent database AND the output of the transaction must also be a consistent database.
- A transaction must execute completely or not at all.

Transaction Management - Motivating Example

A customer transfers $50 from savings account to chequing account.

- 1) The banking system deducts $50 from savings.
- 2) The system crashes before adding $50 to chequing.

Where has the $50 gone?

Without a transaction, the $50 could be lost!
The customer wanted the withdraw and deposit to both happen in one step, or neither action to happen.



ACID Properties

To preserve integrity, transactions have the following properties:

- ***Atomicity*** **-** Either all operations of the transaction are properly reflected in the database or none are.
- ***Consistency*** **-** Execution of a transaction in isolation preserves the consistency of the database.
- ***Isolation*** **-** Controls how concurrent transactions interact. At the strongest level, transactions behave as though they execute one at a time and are unaware of other concurrently executing transactions.
- ***Durability*** **-** After a transaction successfully completes, the changes it has made to the database persist, even if there are system failures.

ACID Properties

Question: A transaction reads an update made by another transaction before it commits. What ACID property is being relaxed?

A) atomicity

B) consistency

C) isolation

D) durability

E) none of them

Transaction Definition in SQL

In SQL, a transaction begins implicitly.

A transaction in SQL ends by:

- **Commit** makes the transaction's changes permanent.
- **Rollback** aborts current transaction and discards its updates. Failures may also cause a transaction to be aborted.

In programming with databases you can often set the connection to **auto-commit** which means a commit is done after every statement.

Example Transactions

Transaction to deposit $50 into a bank account:

```
-- TRANSACTION T1
UPDATE Account SET balance=balance+50 WHERE num = 'S1';
COMMIT;
```

Transaction to calculate totals for all accounts (twice):

```
-- TRANSACTION T2:
SELECT SUM(balance) as total1 FROM Account;
SELECT SUM(balance) as total2 FROM Account;
COMMIT;
```

Transaction to add a new account:

```
-- TRANSACTION T3:
INSERT INTO ACCOUNT (num, balance) VALUES ('S5', 100);
COMMIT;
```

Isolation Levels in SQL

An ***isolation level*** controls how a transaction views the results of other transactions. Lowering the isolation level may improve performance but may cause concurrency issues. Specify an isolation level by:

`SET TRANSACTION ISOLATION LEVEL` **X** where **X** is

- **Serializable -** transactions behave like executed one at a time.
- **Repeatable read - v**alues already read cannot change because of concurrent transactions, but new rows (phantoms) may appear.
- **Read committed -** only committed values can be read, but successive reads may return different values (a non-repeatable read).
- **Read uncommitted** - even uncommitted records may be read. Reading an uncommitted value is called a *dirty read*.

Scheduling of Transactions

Each transaction is a logical unit of database work.

Multiple transactions may execute concurrently.

Depending on the DBMS, transactions may execute in threads or processes scheduled by the operating system.

Transaction operations may be interleaved, producing different **schedules**, while preserving the order within each transaction.

The DBMS uses **concurrency-control protocols** to restrict interleavings and enforce the selected isolation level.

Locking is one possible concurrency-control mechanism.

Transaction Schedule Example

Transaction **T2** that does two queries has its statements interleaved with transaction **T1** that does an update:

```
SELECT SUM(balance) as total1 FROM Account;  --T2
UPDATE Account SET balance=balance+50 WHERE num='S1';--T1
COMMIT;  -- T1
SELECT SUM(balance) as total2 FROM Account; -- T2
COMMIT; -- T2
```

With isolation level ***read committed***, `total1` may not be the same as `total2` because T2 can see T1's committed update.

With isolation level ***repeatable read***, **T2** cannot observe any changes to values it has already read, so the two totals must be equal.

Transaction Schedule Example (2)

Transaction **T2** that does two queries has its statements interleaved with transaction **T3** that does an insert:

```
SELECT SUM(balance) as total1 FROM Account;  --T2
INSERT INTO ACCOUNT (num, balance) VALUES ('S5' , 100);
COMMIT;  -- T3
SELECT SUM(balance) as total2 FROM Account; -- T2
COMMIT; -- T2
```

With isolation level ***repeatable read***, newly inserted rows may appear as phantoms, so `total1` may not be the same as `total2`.

With isolation level ***serializable***, **T2** cannot observe new rows inserted by **T3** between its queries.

Summary of Isolation Levels

Isolation Level	Dirty Reads	Non-repeatable Reads	Phantom Reads
Serializable	No	No	No
Repeatable read	No	No	Possible
Read committed	No	Possible	Possible
Read uncommitted	Possible	Possible	Possible

Trade-off: Stronger isolation prevents more concurrency anomalies but may increase blocking, resource use, or transaction retries.

Read Committed Question

Question: Will this transaction always see the same results for both queries if executed using **READ COMMITTED**?

```
SELECT SUM(balance) as total1 FROM Account;
SELECT SUM(balance) as total2 FROM Account;
COMMIT;
```

A) yes

B) no

Repeatable Read Question

Question: Will this transaction always see the same results for both queries if executed using **`REPEATABLE READ`**?

```
SELECT SUM(balance) as total1 FROM Account;
SELECT SUM(balance) as total2 FROM Account;
COMMIT;
```

A) yes

B) no

JDBC Transaction Example

```
try (Connection con = DriverManager.getConnection(url, uid, pw)) {
    con.setAutoCommit(false);

    try (Statement stmt = con.createStatement()) {
        stmt.executeUpdate(
            "UPDATE Account SET balance=balance-50 WHERE num='S1'");
        stmt.executeUpdate(
            "UPDATE Account SET balance=balance+50 WHERE num='C1'");

        con.commit();
    } catch (SQLException ex) {
        con.rollback();
        throw ex;
    }
} catch (SQLException ex) {
    System.err.println(ex.getMessage());
}
```

Force explicit commit/rollback

Commit work to DB

Rollback work done

Recursive Queries
Using Common Table Expressions (CTEs)

General form:

```
WITH RECURSIVE tableName(attr1,attr2,..attrN) AS (
    <SELECT query that defines recursive relation> )
<SELECT query that uses recursive relation>
```

Example: Return all employees supervised by 'J. Jones'.

```
WITH RECURSIVE supervises(supId,empId) AS (
   SELECT supereno, eno FROM emp WHERE supereno IS NOT NULL
   UNION ALL
   SELECT S.supId, E.eno
   FROM supervises S, emp E
   WHERE S.empId = E.supereno )
SELECT E.eno, E.ename FROM supervises, emp AS S, emp E
WHERE S.ename = 'J. Jones' and supervises.supId = S.eno
       and supervises.empId = E.eno;
```

Conclusion

A ***transaction*** is an ***atomic*** program that executes on the database and preserves the consistency of the database.

ACID properties describe reliable transaction execution.

Isolation levels control which effects of concurrent transactions are visible. SQL isolation levels: serializable, repeatable read, read committed, read uncommitted.

Stronger isolation prevents more concurrency anomalies but may reduce performance or concurrency.

Objectives

- Define: transaction
- List ACID properties.
- Compare the four isolation levels and identify dirty reads, non-repeatable reads, and phantom reads.
- Be able to use WITH RECURSIVE for recursive SQL queries.


UBC
THE UNIVERSITY OF BRITISH COLUMBIA